GEORG-AUGUST-UNIVERSITÄT
GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

# Introduction

Roland Leißa
(based on slides by S. Hack)

University of Göttingen

# Compilers boring?

# Why take a Compiler Course?

- Compilers are everywhere!

  *web browsers, graphics drivers, databases, phones, etc.*
- Compilers are interesting!

  *We'll use concepts from automata, graph theory and algorithms, lattice theory, etc.*
- Learn the foundations of Syntax Analysis

  *Helpful when you design your own language or need to read different types of file format*
- Learn the foundations of Program Analysis

  *Helpful beyond compilers (security, verification, PL)*
- Improve your software engineering skills

  *Compilers are sophisticated artifacts that are hard to test and debug*
- Very good job market for compiler experts!

# Course Organization

- Course materials, etc: Stud.IP
- Matrix Chat!
- Lectures/Exercises: Tue 12–14, Wed 10–12
- Tutorial **(only on demand)**: Tue 14–16
  Ask in Matrix chat or `al-harith.farhad@uni-goettingen.de`
- Compiler Lab: Thur, 12–14
- Team up in groups of up to 3 students
- You need 50% points for the exam

# Lecture

- Lexing
- Parsing
- Semantic Analysis, Type Checking
- Program Analysis and Optimization
- SSA
- Instruction Selection
- Scheduling
- Register Allocation

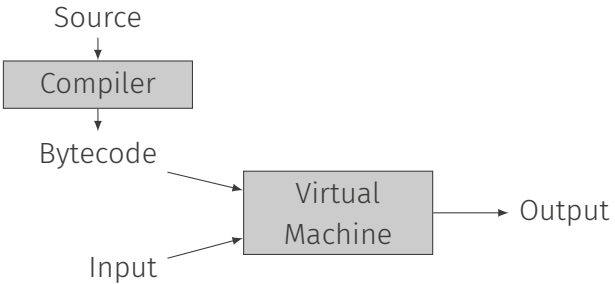
# Basic Notions

# Interpreter

- Reads in source code statement after statement and executes them directly
- Predominant for command languages and languages with REPL (read execute print loop)
- Shells, Matlab, Julia, Python
- Efficient if each statement is only processed once
- No pre-computation necessary

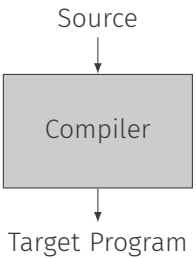
# Virtual Machine

- Analysis of source code and transformation into a more appropriate form to make execution faster, e.g. by
  - Relating definitions and uses of identifiers
  - Type checking if possible
  - Translation into control-flow graphs, stack machine code, etc.
- Code looks machine-like but still has high-level constructs (e.g. method calls, struct access)
- Examples: Python Bytecode, Java Bytecode



# AOT (Ahead-of-Time) Compilation

- Translation to machine code
- Code to interact with runtime system
- Machine code program also defines “virtual” machine:
  - Instruction set architecture of the target machine
  - Operating system (I/O)
  - Runtime system (I/O, memory management)
- Examples: C-Compiler (GCC, LLVM, ...), Rust, Go, etc.



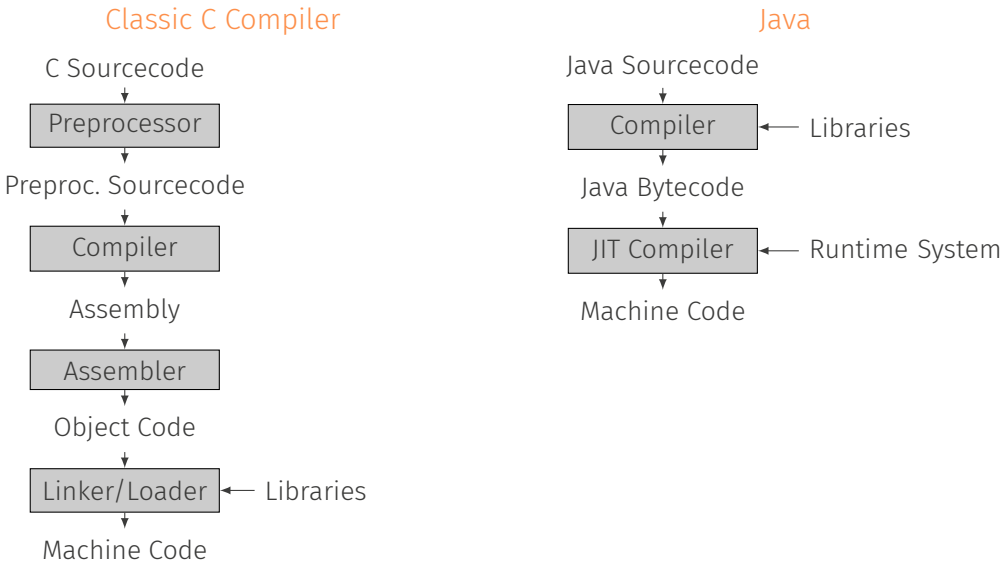

# JIT (Just-in-Time) Compilation

- Compilation during program execution. Typically on-demand and only needed parts.
  - Generated code is much faster than interpretation
  - Dynamic optimization possible (and necessary for some languages): Optimize code based on runtime information
  - Different levels of optimization and compilers depending on how “hot” the code is

## Examples

- .NET Virtual Machine
- Java Virtual Machine
- JavaScript implementations

# The Bigger Picture ...



# Goals of a Compiler

A compiler translates a program from language *S* to language *T* while preserving its semantics and thereby implements *S* in *T*.

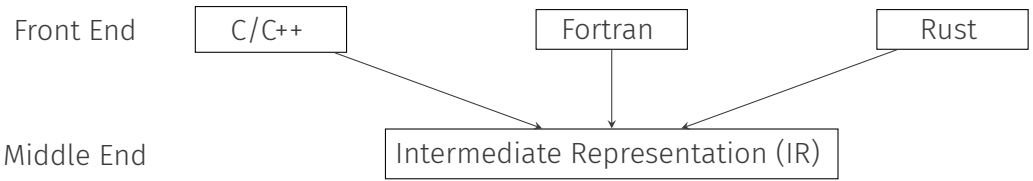
Criteria:

- Correctness
- Efficient resource usage of target program:
  Mostly time and memory
- Low compile time
- Compatibility with other compilers
- Support the programmer:
  Good error messages, warnings, interaction with debugger, profilers, etc.

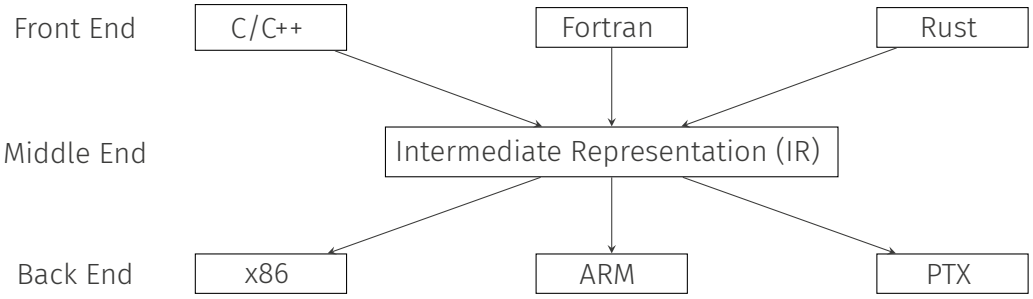
# Architecture & Phases of a Compiler

# Architecture

# Architecture



# Architecture



# Running Example

```
float f(float in) {
    float out;
    out = 2 * in;
    return out;
}
```

# Running Example

```
float f(float in) {
    float out;
    out = 2 * in;
    return out;
}
```

# Lexical Analysis

- **Input:** Sequence of Characters

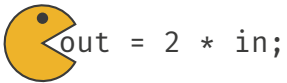
# Lexical Analysis

- **Input:** Sequence of Characters
- **Tasks:**
  - Deconstruct incoming text into meaningful chunks called Tokens
  - Discard useless characters (whitespace, comments)

# Lexical Analysis

- **Input:** Sequence of Characters
- **Tasks:**
  - Deconstruct incoming text into meaningful chunks called Tokens
  - Discard useless characters (whitespace, comments)
- **Output:** Sequence of tokens

# Lexical Analysis



# Lexical Analysis

o  ut = 2 * in;

# Lexical Analysis

ou  t = 2 * in;

# Lexical Analysis

⟨id, out⟩  `= 2 * in;`

# Lexical Analysis

⟨id, out⟩  = 2 * in;

# Lexical Analysis

⟨id, `out`⟩ ⟨`=`⟩  `2 * in;`

# Lexical Analysis

⟨id, out⟩ ⟨=⟩  `2 * in;`

# Lexical Analysis

⟨id, `out`⟩ ⟨`=`⟩ ⟨lit, 2⟩  `* in;`

# Lexical Analysis

⟨id, `out`⟩ ⟨`=`⟩ ⟨lit, 2⟩  `* in;`

# Lexical Analysis

⟨id, `out`⟩ ⟨`=`⟩ ⟨lit, 2⟩ ⟨`*`⟩  `in;`

# Lexical Analysis

⟨id, `out`⟩ ⟨`=`⟩ ⟨lit, 2⟩ ⟨`*`⟩  `in;`

# Lexical Analysis

⟨id, `out`⟩ ⟨`=`⟩ ⟨lit, 2⟩ ⟨`*`⟩ `i`  `n;`

# Lexical Analysis

⟨id, `out`⟩ ⟨=⟩ ⟨lit, 2⟩ ⟨*⟩ ⟨id, `in`⟩ 

# Lexical Analysis

⟨id, `out`⟩ ⟨=⟩ ⟨lit, 2⟩ ⟨*⟩ ⟨id, `in`⟩ ⟨;⟩

# Syntax Analysis

- **Input:** Sequence of Tokens

# Syntax Analysis

- **Input:** Sequence of Tokens
- **Tasks**
  - Checks if input adheres to context-free grammar of source language
  - Detects hierarchical structure of input

# Syntax Analysis

- **Input:** Sequence of Tokens
- **Tasks**
  - Checks if input adheres to context-free grammar of source language
  - Detects hierarchical structure of input
- **Output:** Abstract syntax tree (AST)

# Syntax Analysis

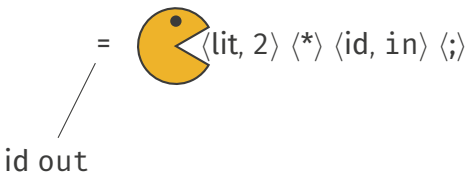
⟨id, `out`⟩ ⟨=⟩ ⟨lit, 2⟩ ⟨*⟩ ⟨id, `in`⟩ ⟨;⟩

# Syntax Analysis

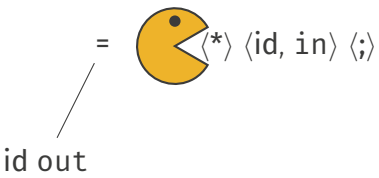
⟨=⟩ ⟨lit, 2⟩ ⟨*⟩ ⟨id, `in`⟩ ⟨;⟩

id `out`

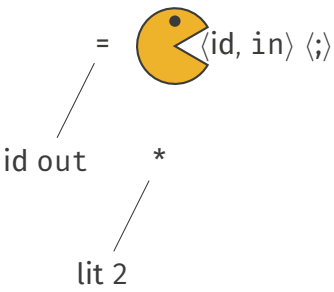
# Syntax Analysis



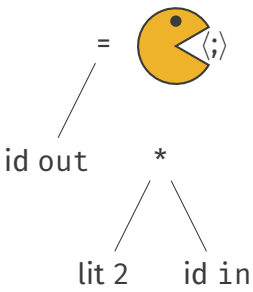
# Syntax Analysis



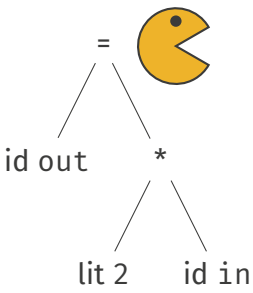
# Syntax Analysis



# Syntax Analysis



# Syntax Analysis



# Semantic Analysis

- **Input:** AST

# Semantic Analysis

- **Input:** AST
- **Tasks:**
  - **Name analysis:** Relate uses to definitions of names
  - **Type analysis:** Elaborate and check the types of entities

# Semantic Analysis

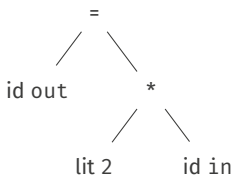
- **Input:** AST
- **Tasks:**
  - **Name analysis:** Relate uses to definitions of names
  - **Type analysis:** Elaborate and check the types of entities
- **Output:** Attributed AST

# Semantic Analysis

```
float f(float in) {
    float out;
    out = 2 * in;
    return out;
}
```

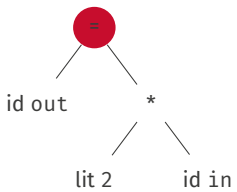
# Semantic Analysis

```
float f(float in) {
    float out;

    return out;
}
```



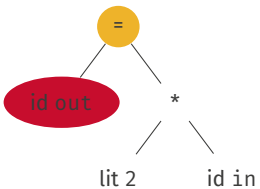
# Semantic Analysis

```
float f(float in) {
    float out;

    return out;
}
```



# Semantic Analysis

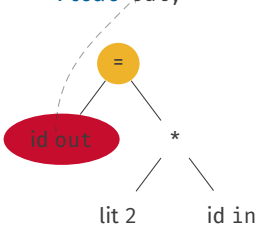
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

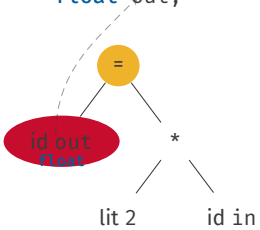
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

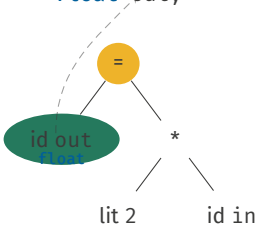
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

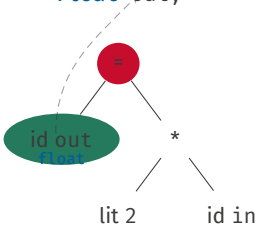
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

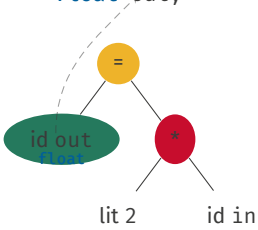
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

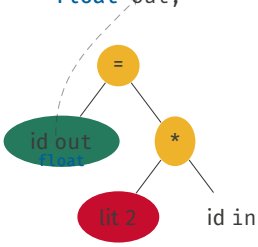
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

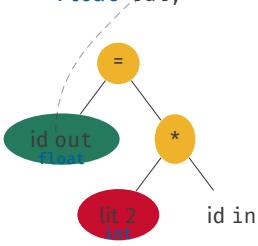
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

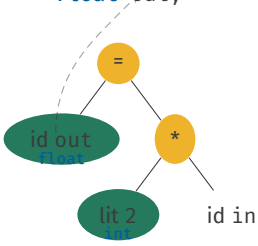
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

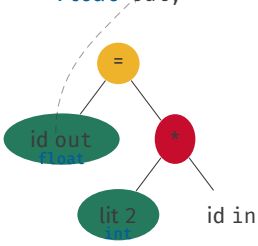
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

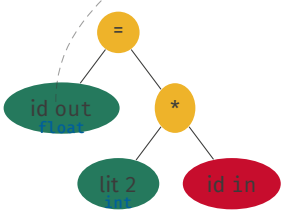
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

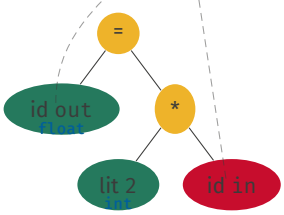
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

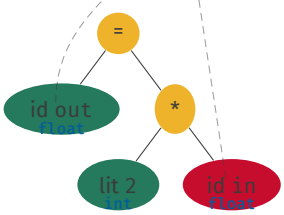
```
float f(float in) {
    float out;
```




```
    return out;
}
```

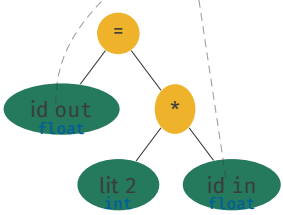
# Semantic Analysis

```
float f(float in) {
    float out;
    return out;
}
```



# Semantic Analysis

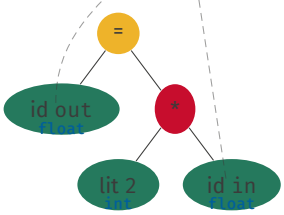
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

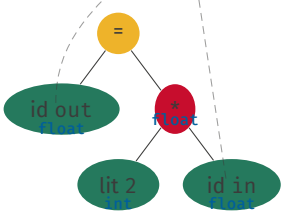
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

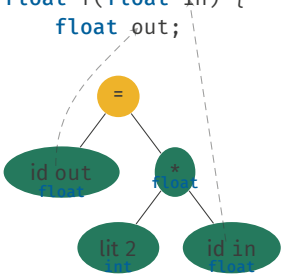
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

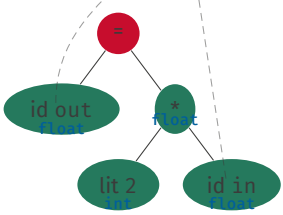
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

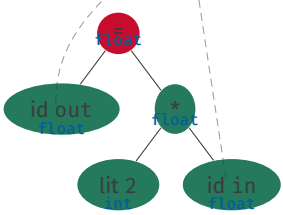
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

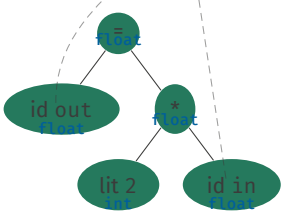
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Semantic Analysis

```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Transformation

- **Input:** Attributed AST

# Transformation

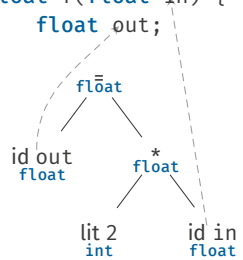
- **Input:** Attributed AST
- **Tasks:** Lower high-level language constructs to target code

# Transformation

- **Input:** Attributed AST
- **Tasks:** Lower high-level language constructs to target code
- **Output:** Machine code or intermediate representation (IR)

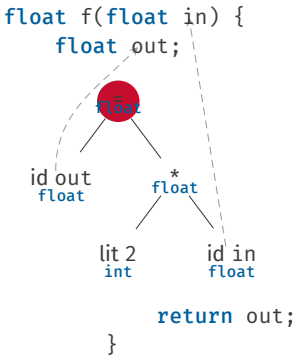
# Transformation

```
float f(float in) {
    float out;
```



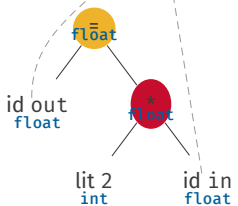

```
    return out;
}
```

# Transformation



# Transformation

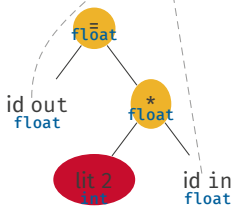
```
float f(float in) {
    float out;
```




```
    return out;
}
```

# Transformation

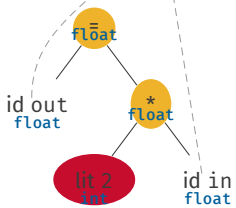
```
float f(float in) {
    float out;
```




```
    return out;
}
```

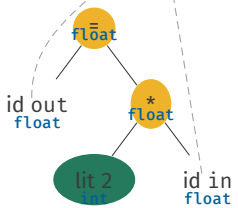
# Transformation

```
float f(float in) {
    float out;
    out = in * 2;
    return out;
}
```



# Transformation

```
float f(float in) {
    float out;
```

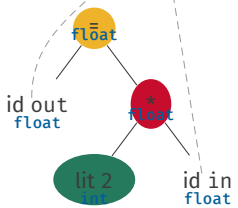



```
    return out;
}
```

2

# Transformation

```
float f(float in) {
    float out;
```

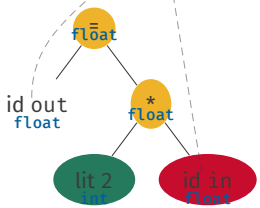



```
    return out;
}
```

```
t1 = inttofloat 2
```

# Transformation

```
float f(float in) {
    float out;
```

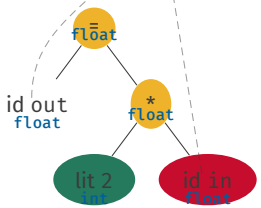



```
    return out;
}
```

```
t1 = inttofloat 2
```

# Transformation

```
float f(float in) {
    float out;
```

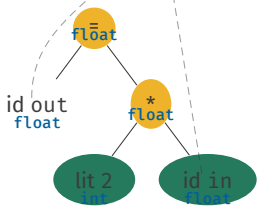



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
```

# Transformation

```
float f(float in) {
    float out;
```

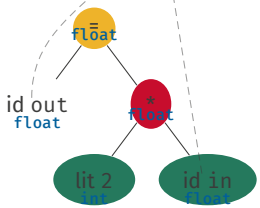



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
```

# Transformation

```
float f(float in) {
    float out;
```

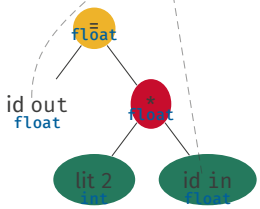



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
```

# Transformation

```
float f(float in) {
    float out;
```

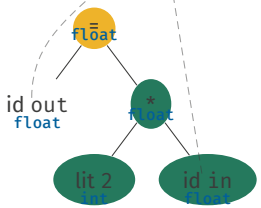



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
```

# Transformation

```
float f(float in) {
    float out;
```

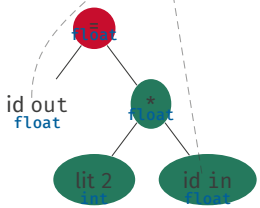



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
```

# Transformation

```
float f(float in) {
    float out;
```

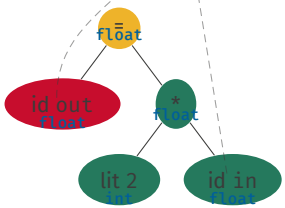



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
```

# Transformation

```
float f(float in) {
    float out;
```

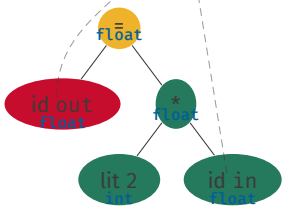



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
```

# Transformation

```
float f(float in) {
    float out;
```

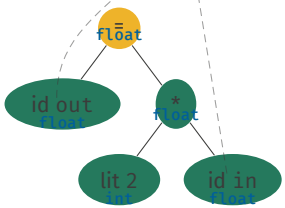



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
          out
```

# Transformation

```
float f(float in) {
    float out;
```

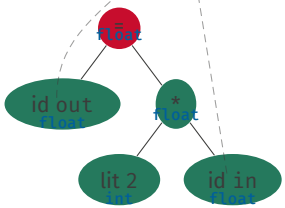



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
          out
```

# Transformation

```
float f(float in) {
    float out;
```

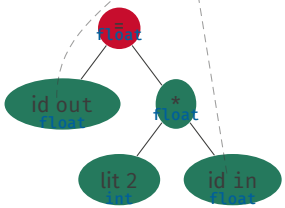



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
          out
```

# Transformation

```
float f(float in) {
    float out;
```

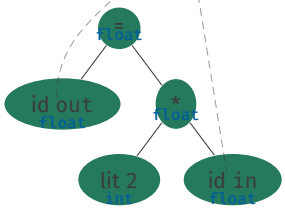



```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
store t3, out
```

# Transformation

```
float f(float in) {
    float out;
```




```
    return out;
}
```

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
store t3, out
```

# IR Example: Java-Bytecode

```
int fib(int x) {
  // base case
  if (x <= 1)
    return x;
  // recursion
  return fib(x-1)
       + fib(x-2);
}
```

```
iload_0              ; Push parameter 0 (x)
iconst_1             ; Push const 1
if_icmpgt label0     ; If larger goto label0
iload_0              ; Push parameter 0
ireturn              ; Back to caller
label0:
iload_0              ; Push parameter 0
iconst_1             ; Push const 1
isub                 ; subtract
invokestatic fib     ; Call method fib
iload_0              ; Push parameter 0
iconst_2             ; Push const 2
isub                 ; subtract
invokestatic fib     ; Call method fib
iadd                 ; add
ireturn              ; Back to caller
```

# IR Example: LLVM Bitcode

```
define dso_local i32 @_Z3fibi(i32 %0) local_unnamed_addr #0 !dbg !7 {
  %2 = icmp slt i32 %0, 2, !dbg !15
  br i1 %2, label %11, label %3, !dbg !17

3:                                                ; preds = %1, %3
  %4 = phi i32 [ %8, %3 ], [ %0, %1 ]
  %5 = phi i32 [ %9, %3 ], [ 0, %1 ]
  %6 = add nsw i32 %4, -1, !dbg !18
  %7 = tail call i32 @_Z3fibi(i32 %6), !dbg !19
  %8 = add nsw i32 %4, -2, !dbg !20
  %9 = add nsw i32 %7, %5, !dbg !21
  %10 = icmp slt i32 %4, 4, !dbg !15
  br i1 %10, label %11, label %3, !dbg !17

11:                                               ; preds = %3, %1
  %12 = phi i32 [ 0, %1 ], [ %9, %3 ]
  %13 = phi i32 [ %0, %1 ], [ %8, %3 ]
  %14 = add nsw i32 %13, %12, !dbg !21
  ret i32 %14, !dbg !22
}
```

# Optimization

- **Input:** IR

# Optimization

- **Input:** IR
- **Tasks:**
  - Reduce resource consumption: execution time, memory
  - Preserve semantics

# Optimization

- **Input:** IR
- **Tasks:**
  - Reduce resource consumption: execution time, memory
  - Preserve semantics
- **Output:** IR

- Optimization has various purposes:
  - to prevent programmers writing bad (“optimized”) code
  - remove overhead from language abstractions that manifests during syntax-directed translation
  - separation of concerns in the compiler:
    some optimizations remove clutter left over by other optimizations

# Optimization

```
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
store t3, out
```

# Optimization

```
t1 = 2.f
t2 = load in
t3 = mul t1, t2
store t3, out
```

# Optimization

```
t2 = load in
t3 = mul 2.f, t2
store t3, out
```

# Optimization

```
t2 = load in
t3 = add t2, t2
store t3, out
```

# Code Generation

- **Input:** IR
- **Tasks:**
  - Instruction selection: use ISA of target arch
  - Scheduling: re-order instructions
  - Register allocation: allocate registers to IR vars
- **Output:** Machine code

# Code Generation

```
t2 = load in
t3 = add t2, t2
store t3, out
```

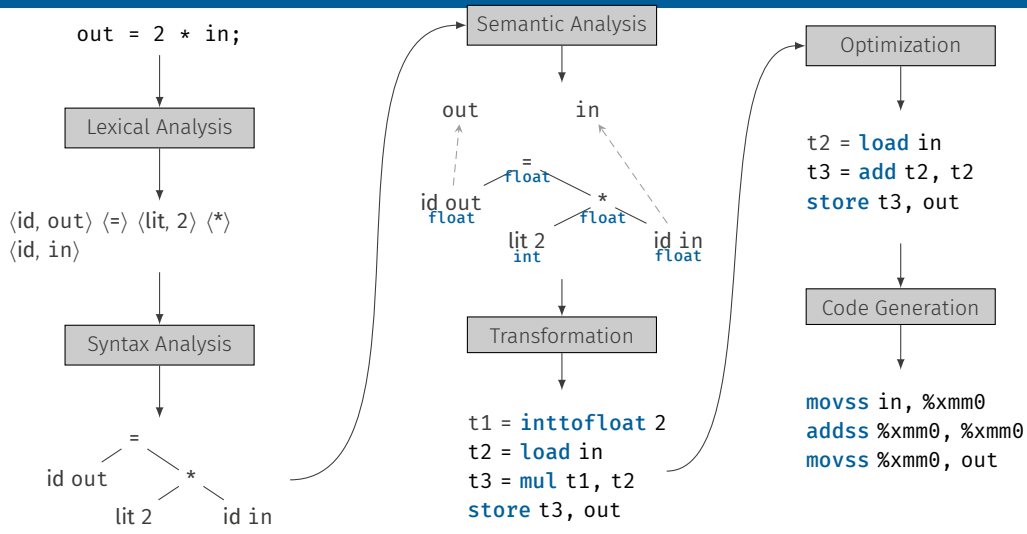
```
movss in, %xmm0
addss %xmm0, %xmm0
movss %xmm0, out
```

# Code Generation: Challenges

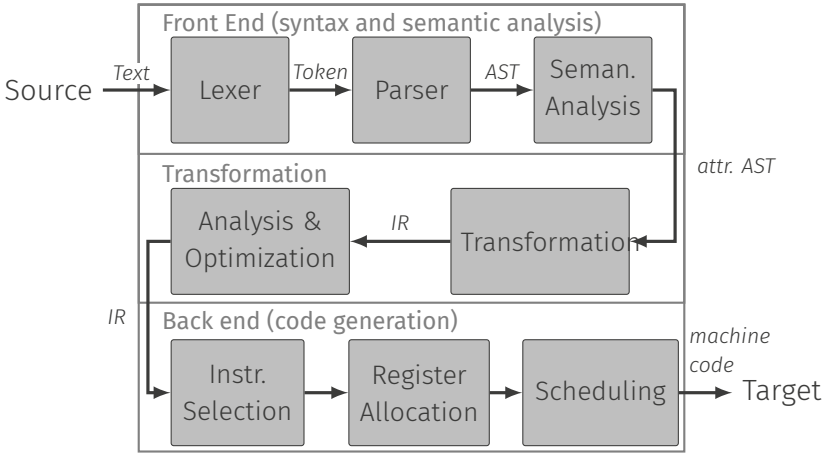
- Compile-time has to be sub-quadratic for the common case
- Most relevant code generation problems are NP-hard
- Mainly because target machines have resource constraints
  (finite amount of memory, registers, parallelism, etc.)

- Target machines become less “standard” (aka heterogeneous)
  Think of GPUs, accelerators or even FPGAs
- Target machines become more complex
  Memory hierarchy, out-of-order execution, speculation, etc.
- Hence it is often impossible to give a precise notion of “optimality” with respect to the quality of the code.
- End of Dennard Scaling:
  Performance $\cong$ Performance / Watt

# Summary


out = 2 * in;
Lexical Analysis
⟨id, out⟩ ⟨=⟩ ⟨lit, 2⟩ ⟨*⟩
⟨id, in⟩
Syntax Analysis
=
id out
*
lit 2
id in
Semantic Analysis
out
in
=
float
id out
float
*
float
lit 2
int
id in
float
Transformation
t1 = inttofloat 2
t2 = load in
t3 = mul t1, t2
store t3, out
Optimization
t2 = load in
t3 = add t2, t2
store t3, out
Code Generation
movss in, %xmm0
addss %xmm0, %xmm0
movss %xmm0, out

# Structure of a Compiler



# Performance

# Optimizations are usually undecidable in General

```
if (does_this_turing_machine_halt(P)) {
    A;
} else {
    B;
}
```

# Example: Optimizing away Abstractions

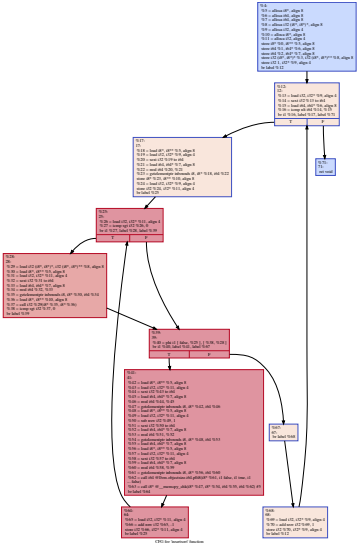
```
typedef int (cmp_t)(void const *a, void const *b);

static void insertsort(void* vals, size_t n,
                       size_t sz, cmp_t *cmp) {
#define V(k) (((char *) vals) + ((k) * sz))
    for (int i = 1; i < n; i++) {
        void *x = V(i);
        for (int j = i; j > 0 &&
             cmp(V(j), x) > 0; j--) {
            memcpy(V(j), V(j - 1), sz);
        }
    }
#undef V
}

static int int_cmp(void const *a, void const *b) {
    int const *p = a;
    int const *q = b;
    return (*q < *p) - (*p < *q);
}

static void insertsort_int(int *vals, size_t n) {
    insertsort(vals, n, sizeof(int), int_cmp);
}
```

```
int main(int argc, char const **argv)
{
    int n = 1000;

    if (argc > 1)
        n = atoi(argv[1]);

    int *arr = malloc(n * sizeof(arr[0]));
    for (int i = 0; i < n; i++)
        arr[i] = rand();
    insertsort_int(arr, n);

    return arr[n - 1];
}
```

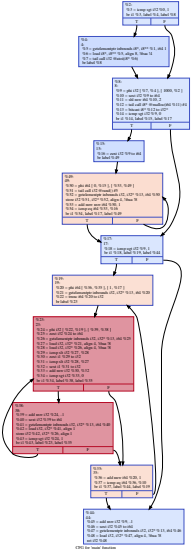
- Inline all functions
- Function pointer to `cmp` becomes constant
- Size of element type becomes constant
- Replace call to `memcpy` to register copy

# Example: Optimizing away abstractions

Unopt: Only `insertsort`



Opt: Entire Program



Speedup:

1.6x on my laptop

# Example: Optimizing Triple Code

## Input:

```
for (i = 0; i < 20; ++i)
  res += a[i] * b[i];
```

## Optimizations:

- Strength Reduction: Adapt stepping of loop var to 4 to remove `mul i, 4`
- Loop Inversion: Move cond jump of loop to the end
- Loop Invariant Code Motion: Move code in front of the loop

## IR:

```
i = 0
loop:
jmpge i, 20, end
aval = load a
idx0 = mul i, 4
addr0 = add aval, idx0
val0 = load addr0
bval = load b
idx1 = mul i, 4
addr1 = add bval, idx1
val1 = load addr1
prod = mul val0, val1
res = add res, prod
i = add i, 1
jump loop
end:
```

## Optimized IR:

```
addr0 = load a
addr1 = load b
addr2 = add addr0, 80
loop:
val0 = load addr0
val1 = load addr1
prod = mul val0, val1
res = add res, prod
addr0 = add addr0, 4
addr1 = add addr1, 4
jmpl addr0, addr2, loop
```

# Some state-of-the-art Optimizations

- Sparse Conditional Constant Propagation
- Jump Threading
- If Conversion
- Constant Folding
- Arithmetic Simplifications
- Induction Variable Elimination
- Strength Reduction
- Devirtualization
- Dead Code Elimination
- Tail Call Elimination
- Function Inlining
- Common Subexpression Elimination
- Dead Store Elimination
- Partial Redundancy Elimination
- Global Value Numbering
- Instruction Combining
- Loop Inversion
- Loop Fusion
- Loop Unrolling
- Loop-Invariant Code Motion
- Loop Unswitching
- Vectorization
- Software Pipelining
- Scalar Replacement
- Block Merging
- Register Allocation
- Scheduling
- Peephole Optimizations
- …

# A Note on Performance

- Textbook implementations typically don't provide performance
- Respecting/exploiting hardware properties important
- Example: Matrix Multiplication

```
for (int i = 0; i < M; i++)
  for (int j = 0; j < N; j++) {
    for (int k = 0; k < P; k++)
      C[i][j] += A[i][k] * B[k][j];
  }
```

Textbook

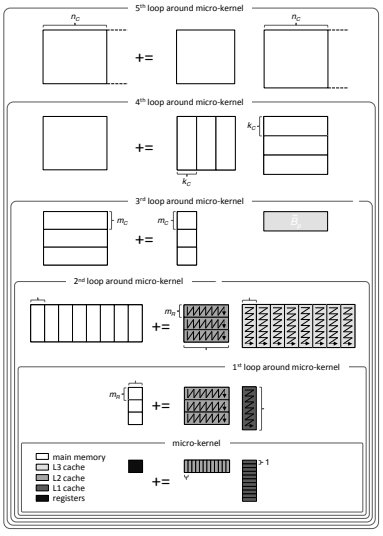
On my notebook: 0.47 GFlops/s

```
for (int i = 0; i < M; i++)
  for (int k = 0; k < P; k++) {
    for (int j = 0; j < N; j++)
      C[i][j] += A[i][k] * B[k][j];
  }
```

Loop interchange

On my notebook: 3.94 GFlops/s

# A Note on Performance (cont'd)




- Zee, van de Geijn, BLIS: A Framework for Rapidly Instantiating BLAS Functionality, TOMS 2015
- The end of the spectrum, hand-tuned implementation
- Multiple levels of loop tiling for cache locality
- Transposition to get more streaming accesses
- Innermost "micro-kernel": Outer-product, hand-written in assembly
- On my notebook: 51.06 GFlops/s